

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 03 -

BOAS PRÁTICAS EM DOCKER PARA ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	11
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13
PALAVRAS-CHAVE.....	14

EMSE

O QUE VEM POR AÍ?

Depois de aprender como construir e executar containers, chegou o momento de evoluir do “funciona na minha máquina” para o “funciona com qualidade de produção”. Nesta aula, vamos aprofundar as decisões que tornam imagens Docker adequadas para ambientes reais de Machine Learning: performance de build, redução de tamanho, rastreabilidade de versões e segurança desde a origem.

Você vai entender como pequenas escolhas no Dockerfile impactam custo de infraestrutura, tempo de deploy e risco operacional. Ao final, terá repertório para avaliar criticamente uma imagem de inferência e aplicar padrões que aumentam confiabilidade em pipelines de MLOps.

HANDS ON

No hands on desta aula, você vai partir de uma imagem inicial de inferência com problemas típicos de projetos em estágio inicial, como excesso de pacotes, ausência de travamento de dependências e execução com privilégios elevados. As videoaulas mostram como refatorar esse cenário, etapa por etapa, até chegar em uma imagem enxuta, reproduzível e pronta para produção.

Durante a prática, você acompanhará a aplicação de multi-stage build, reorganização de camadas para melhor uso de cache e introdução de medidas de hardening. O foco é observar o efeito de cada mudança em indicadores concretos, como tamanho final da imagem, tempo de build e superfície de vulnerabilidade.

Ao final, você terá um comparativo técnico entre “antes” e “depois”, com critérios objetivos para decidir quando uma imagem está pronta para seguir na esteira de CI/CD. Essa abordagem ajuda a transformar boas práticas em padrão operacional da equipe, não apenas em recomendações pontuais.

SAIBA MAIS

Boas práticas como fundamento de confiabilidade em MLOps

Em projetos de Machine Learning, o Dockerfile é parte da arquitetura de produção. Ele influencia tempo de build, segurança, custo de infraestrutura, velocidade de release e capacidade de rollback. Um container “funcional” não é necessariamente um container “operável”.

A diferença entre esses dois estados costuma aparecer quando o time precisa publicar novas versões de modelo com frequência. Sem padrão técnico, cada release vira risco operacional.

Multi-stage build além da otimização de tamanho

A estratégia de multi-stage build separa claramente etapas de construção e execução. No estágio de build, você instala ferramentas e dependências temporárias; no estágio final, mantém apenas runtime e componentes essenciais para inferência.

Esse desenho gera três ganhos práticos:

- Menor superfície de ataque.
- Imagem mais leve para distribuição.
- Menor complexidade de manutenção em produção.

Exemplo de `Dockerfile` com multi-stage para API de inferência:

```
FROM python:3.11-slim AS build
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r
requirements.txt

FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=build /install /usr/local
COPY app/ ./app
EXPOSE 8000
```

```
CMD ["python", "app/main.py"]
```

Código-fonte 1 – Exemplo de `Dockerfile` com multi-stage para API de inferência
Fonte: Elaborado pelo autor (2026)

A imagem base deve equilibrar compatibilidade e segurança. Em ML, bases “minimalistas demais” podem quebrar bibliotecas nativas; bases amplas demais aumentam custo e risco.

CRITÉRIO	BASE AMPLA	BASE ENXUTA	EFEITO EM PRODUÇÃO
Tamanho	Alto	Menor	Rollout rápido
CVEs potenciais	Maior	Menor	Baixo risco de bloqueio
Compatibilidade	Alta	Validada	Menos rollback
Manutenção	Difusa	Padronizada	Melhor governança

Tabela 1 – Escolha de imagem base com critério técnico
Fonte: Elaborado pelo autor (2026)

Camadas e cache como economia contínua

A ordem das instruções no Dockerfile afeta diretamente o cache. Dependências estáveis devem ser instaladas antes da cópia do código da aplicação. Isso evita reconstrução de camadas pesadas em alterações pequenas.

Em pipelines de ML com múltiplas execuções diárias, essa otimização reduz custo e acelera feedback para a equipe.

```
FROM python:3.11-slim
WORKDIR /app
```

```
# Camada estável primeiro
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Código muda com mais frequência
COPY . .
CMD ["python", "app/main.py"]
```

Código-fonte 2 – Exemplo comparativo de ordenação de camadas para melhor cache (Shell)
Fonte: Elaborado pelo autor (2026)

Reprodutibilidade forte: dependências travadas

Sem lock de dependências, o mesmo Dockerfile pode gerar comportamentos diferentes ao longo do tempo. Em inferência, isso significa risco de alteração silenciosa de resultado.

Boas práticas mínimas:

- Pinning de versão.
- Lock de resolução.
- Tag imutável.
- Vínculo entre imagem e versão de modelo.

Mini-case - regressão silenciosa por atualização transitiva: um serviço manteve o mesmo código, mas mudou o resultado de predição após a atualização automática de biblioteca secundária. A ausência de lock dificultou o diagnóstico. Após a padronização de dependências travadas, o problema deixou de se repetir.

Versionamento semântico aplicado a artefatos de ML

Versionar imagens como `major.minor.patch` (Semantic Versioning) melhora a comunicação e reduz a ambiguidade:

- `major`: quebra de contrato.
- `minor`: adição sem quebra.
- `patch`: correção sem impacto externo relevante.

Mudança de schema da API > major > Entrada obrigatória alterada |

Novo endpoint de inferência > menor > Nova rota funcional |

Correção de parsing/log > patch > Ajuste sem quebra de contrato.

Segurança por padrão: build e runtime

Segurança não deve ser etapa final. Ela precisa estar no fluxo desde o Dockerfile:

- Execução com usuário não root.
- Redução de pacotes instalados.
- Restrição de permissões.
- Scanner obrigatório no pipeline.

Em ML, stacks amplas aumentam exposição a CVEs. Por isso, políticas de bloqueio por severidade são fundamentais.

Exemplo de `Dockerfile` com hardening básico (usuário não root):

```
FROM python:3.11-slim
WORKDIR /app

RUN addgroup --system mlgroup && adduser --system --ingroup
mlgroup mluser
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

RUN chown -R mluser:mlgroup /app
USER mluser

EXPOSE 8000
CMD ["python", "app/main.py"]
```

Código-fonte 3 – Exemplo comparativo de ordenação de camadas para melhor cache (Shell)

Fonte: Elaborado pelo autor (2026)

`.dockerignore` como proteção de dados

O arquivo `.dockerignore` evita incluir arquivos desnecessários no contexto de build, como datasets locais, notebooks, caches e credenciais. Além de acelerar build, reduz risco de vazamento.

Essa prática é simples e frequentemente negligenciada, apesar de ser uma das medidas de segurança com melhor custo-benefício.

```
# arquivos de experimento
*.ipynb
notebooks/

# artefatos locais e caches
__pycache__/
*.pyc
.pytest_cache/

# dados e modelos locais não versionados
data/
models/

# credenciais e variáveis sensíveis
.env
secrets/
```

Código-fonte 4 – Exemplo de `.dockerignore` alinhado a projetos de ML (Bash)
Fonte: Elaborado pelo autor (2026)

Operabilidade: imagem pronta para sustentação

Uma imagem de produção precisa nascer pronta para operação:

- Logs estruturados.
- Healthcheck coerente.
- Configuração externa.
- Mensagens de erro acionáveis.

Sem esses elementos, o troubleshooting em produção se torna lento e dependente de conhecimento tácito.

Mini-case - template de Dockerfile no nível de squad: após incidentes recorrentes, uma equipe criou um template padrão com checklist obrigatório: multi-stage, usuário não root, lock de dependências, healthcheck e scanner. O resultado foi redução de retrabalho em revisão e maior previsibilidade de deploy entre times.

Erros recorrentes e mitigação

Erro comum 1: imagem excessivamente grande.

Mitigação: multi-stage, revisão de base e limpeza de contexto.

Erro comum 2: uso indiscriminado de `latest`.

Mitigação: tags imutáveis e política de promoção por ambiente.

Erro comum 3: dependências sem lock.

Mitigação: pinning rigoroso e validação em pipeline.

Erro comum 4: configuração crítica embutida no código/imagem.

Mitigação: externalização por variáveis e secrets.

Boas práticas em Docker para ML formam um sistema integrado de qualidade. Quando build, versionamento, segurança e operabilidade são padronizados, o time ganha velocidade com controle, reduz risco de regressão e melhora confiabilidade de produção.

MERCADO, CASES E TENDÊNCIAS

Uma tendência consolidada no mercado é tratar imagem Docker como artefato crítico de engenharia, com métricas de qualidade e segurança antes da promoção para produção. Esse movimento cresce principalmente em empresas com alto volume de deploy, onde pequenos problemas de build se multiplicam em custo e risco operacional.

Um caso real é o da ****The Warehouse Group****, empresa de varejo da Nova Zelândia, que reportou ganhos expressivos ao padronizar sua estratégia com Docker, incluindo redução do tempo de feedback técnico e ganhos relevantes de produtividade em escala. O relato oficial pode ser consultado [aqui](#).

Em paralelo, cresce o uso de políticas automatizadas para bloquear imagens sem requisitos mínimos de segurança, além da adoção de imagens base internas homologadas. A tendência para times de ML é clara: eficiência e segurança da imagem deixam de ser etapa final e passam a ser critério de entrada no processo de release.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu a transformar Dockerfiles de ML em artefatos de produção: menores, mais rápidos, versionados e seguros. Viu como multi-stage builds, minimização de camadas, versionamento semântico e hardening básico reduzem risco e aumentam previsibilidade operacional.

EMEND

REFERÊNCIAS

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes**: Up and Running. Sebastopol: O'Reilly Media, 2022.

MERKEL, D. **Docker**: Lightweight Linux Containers for Consistent Development and Deployment. Houston: Belltown Media, 2014.

POULTON, N. **Docker Deep Dive**. [S.l.]: Independently Published, 2023.

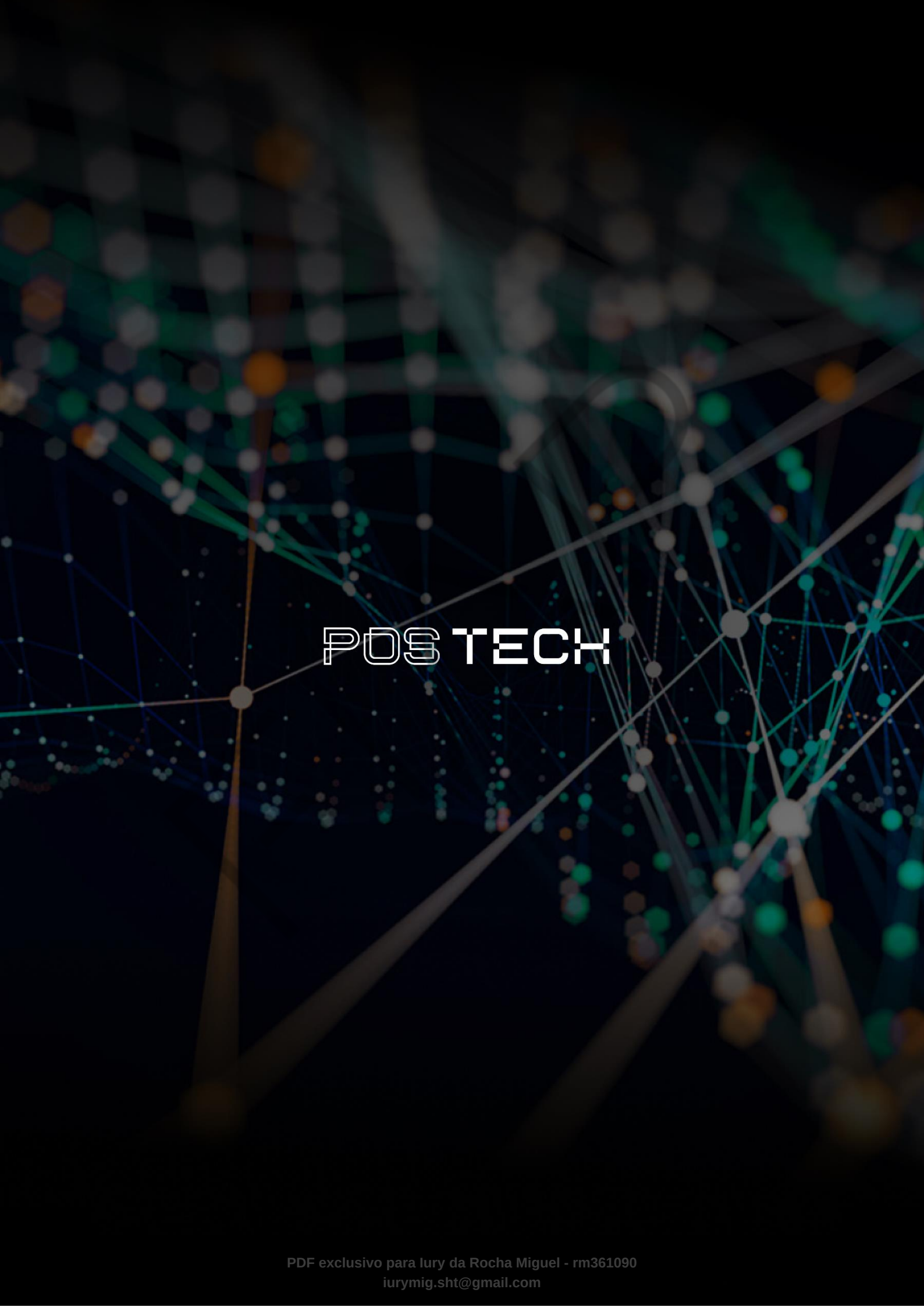
SATO, D. **DevOps na prática**: Entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.

VERAS, M. **Virtualização**: Da teoria a soluções práticas. São Paulo: Novatec, 2019.

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMEND



POSTECH